

Talos RAG

Owner's Manual

How the retrieval system is configured, built, used, operated, and backed by evaluation.

The whole system in one sentence. One pipeline, driven by one config object (`RagConfig`), exposed through one endpoint (`/ask`), and measured by an eval harness that calls the **same** pipeline code. Files and chat messages live as vectors in one Milvus collection (`talos_documents`), told apart by a source field. Master three chokepoints — **RagConfig** (all knobs), **build_rag_pipeline** (all retrieval), **RagTrace** (all observability) — and you can scale, debug, and fix any part of it.

Branch `feature/chat-message-memory` · worktree `~/talos-main` · src/rag/

1 · System overview

Talos RAG has exactly one flow: **writers** put vectors into a single Milvus collection; **readers** query them to answer a question. There is no second pipeline hiding anywhere — even the evaluation harness reads through the same core (bottom of the diagram).

1 · System overview — one collection, writers on the left, readers on the right

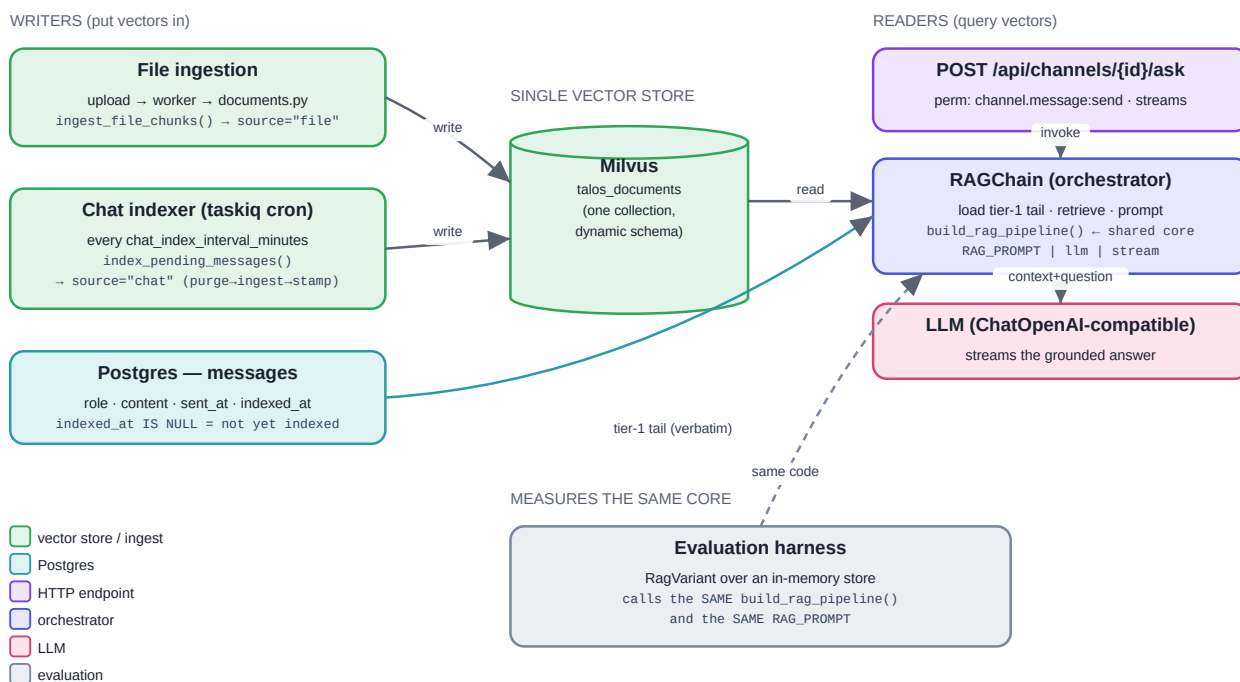


Diagram 1: Writers on the left, one vector store in the middle, readers on the right. Evaluation shares the core.

Key idea. Everything funnels through `talos_documents`. File chunks are tagged `source="file"`; chat-memory chunks are tagged `source="chat"`. Retrieval filters on that field, so the two streams never bleed into each other.

There is only one RAG entry point in the app — `POST /api/channels/{id}/ask`. Ordinary chat (`POST /channels/{id}/messages`) just stores and broadcasts a message; it never calls the LLM. So `/ask` is additive and self-contained.

2 · Configuration — the single source of truth

Every knob is a field on `RagConfig` in `src/config/config.py`, instantiated once as `global_rag_config`. It reads from environment variables (and `.env`); the field name uppercased **is** the variable name. There is no YAML for RAG config. To change behaviour you set an env var, or change the default here — that is the only place.

Field (env var)	What it does	Default
<code>openai_model</code>	Answer + query-rewrite LLM	<code>gpt-4o-mini</code>
<code>embedding_provider</code> / <code>embedding_model</code>	How text becomes vectors	<code>openai</code> / <code>text-embedding-3-small</code>
<code>milvus_collection_name</code>	The one collection (app + CLI agree)	<code>talos_documents</code>

Field (env var)	What it does	Default
use_query_rewrite	Rewrite the query before retrieval (1 LLM call)	True
use_hyde	Hypothetical-document embedding for the query (1 LLM call)	True
use_reranking	Cross-encoder reranks candidates	True
rerank_fetch_k	Candidate pool fetched before reranking down to top-k	20
use_hybrid_retrieval	Add BM25 lexical channel (needs a corpus)	False
compression_type / ..._similarity_threshold	Post-retrieval context compression	none / 0.76
retrieval_top_k	Final chunks fed to the prompt	5
chat_recall_k / chat_context_cap	Tier-2 recall size / tier-1 tail cap	3 / 50
chat_index_interval _minutes / _grace_seconds	Indexer cadence / settle window	5 / 300

Key idea. The config isn't just read at startup — it is **injected** through every factory (config=). That is what lets one process run several configs at once, which is exactly how evaluation works (Diagram 5A).

3 · Component map

Everything lives under src/rag/ (plus the indexer in src/processing/). Each file has one job.

File	Responsibility
config/config.py	RagConfig — all knobs; global_rag_config
rag/vector_store.py	Milvus connection, get_embeddings (cached), get_workspace_vectorstore, deletes, WORKSPACE_COLLECTION
rag/ingestion.py	ingest_file_chunks (source=file), ingest_chat_messages (source=chat), format_citations
rag/retrieval/retrievers.py	build_rag_pipeline — the shared retrieval composition
rag/retrieval/query_processing.py	get_query_rewriter, get_hyde_embeddings
rag/retrieval/compression.py	compression_retriever
rag/generation.py	get_llm, get_memory
rag/rag_chain.py	RAGChain — orchestrator; retrieve → prompt → stream; fills the trace
rag/trace.py	RagTrace — the observability record
rag/router.py	POST /ask endpoint; loads the tier-1 tail
processing/chat_indexing.py	index_pending_messages — the cron indexer

4 · Lifecycle of one /ask request

`RAGChain.stream_query` is the spine. The router loads the recent un-indexed tail (tier 1) and the message ids, persists the question, constructs the chain, and streams. Retrieval pulls file chunks **and** this channel's remembered chat, then the prompt is assembled and the LLM streams tokens. Afterwards the trace is filled and the assistant answer is persisted.

2 · Lifecycle of one /ask request

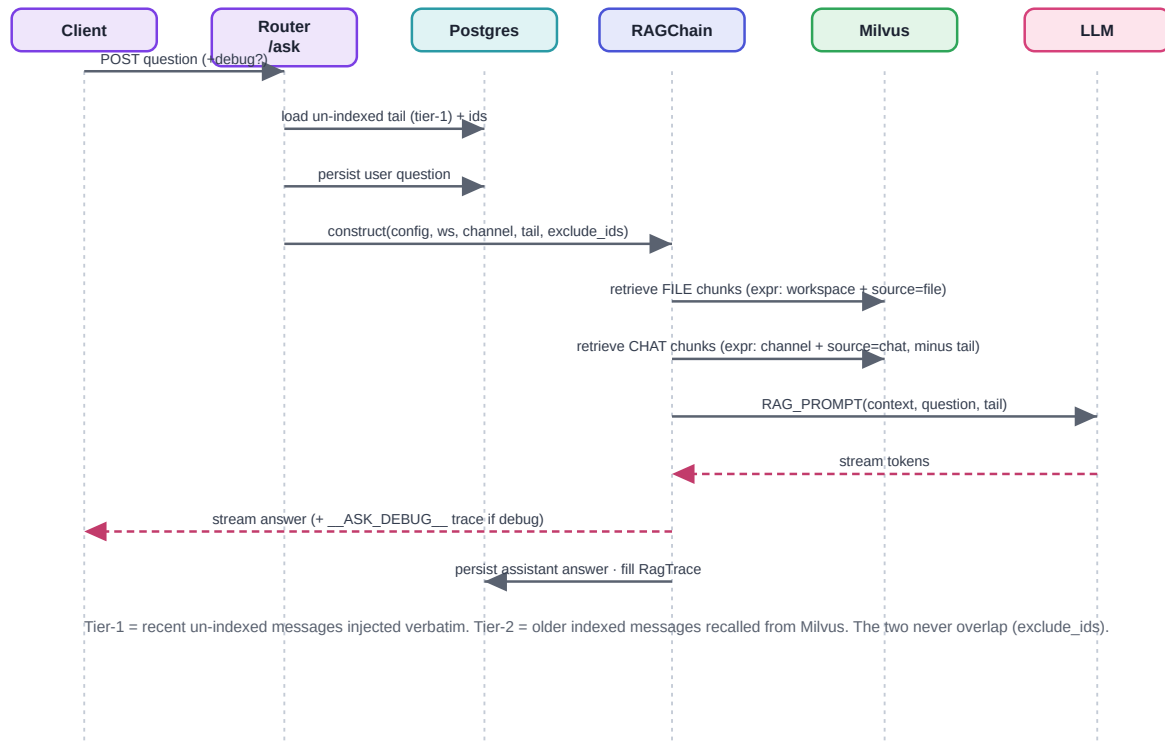


Diagram 2: One request, top to bottom. Dashed red = streaming back to the caller.

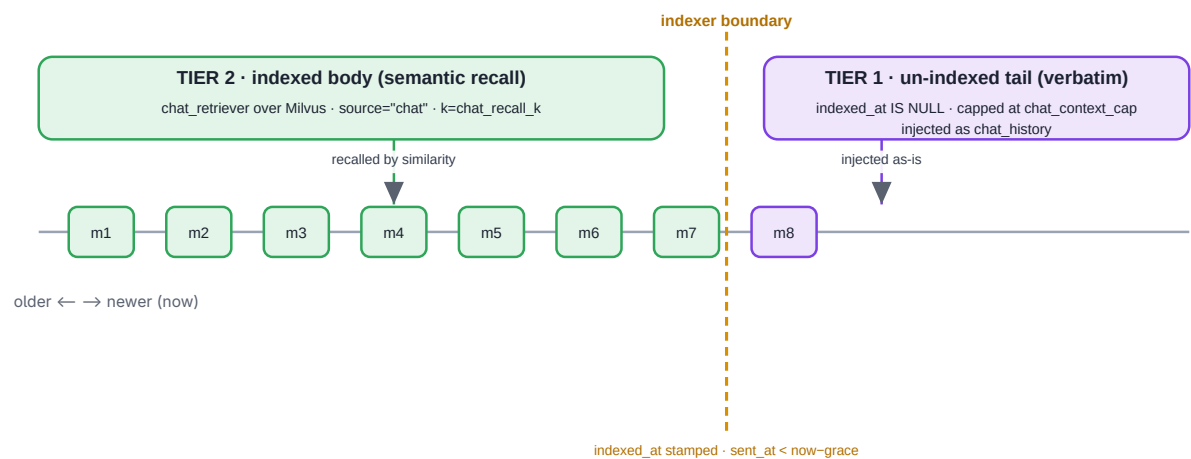
5 · Two-tier chat memory

This is the one non-obvious idea, so hold it clearly. A channel's conversation is split at the **indexer boundary**:

- **Tier 1** — recent messages the indexer hasn't touched yet (`indexed_at IS NULL`). Injected into the prompt **verbatim**, capped at `chat_context_cap`.
- **Tier 2** — older messages the indexer has embedded into Milvus. Recalled **semantically** by `chat_retriever` (scoped to the channel, `source="chat"`).

A message is in exactly one tier. The router passes the tail's ids as `exclude_message_ids`, so a message momentarily on both sides (its vector is in Milvus a beat before its `indexed_at` commit lands) is still counted once.

3 • Two-tier chat memory — every message is in exactly one tier



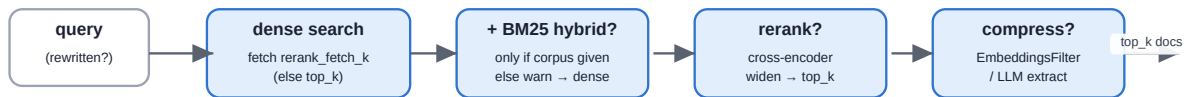
The indexer moves messages left across the boundary (stamps indexed_at). exclude_message_ids drops any that are momentarily on both sides.

Diagram 3: The indexer stamps messages and moves them from tier 1 (verbatim) to tier 2 (semantic recall).

6 • build_rag_pipeline() — the shared retrieval core

All retrieval logic lives in one function. Read it once and you understand every retrieval path — production and evaluation both call it.

4 • build_rag_pipeline() — the one shared retrieval composition



Each stage is a RagConfig toggle: use_hybrid_retrieval · use_reranking (+rerank_fetch_k) · compression_type (+threshold).
PROD passes a Milvus store + tenant filter (search_kwargs). EVAL passes an in-memory store + corpus (so hybrid actually runs). Same function.

Diagram 4: Dense → optional hybrid → optional rerank (with widening) → optional compression. Each stage is a config toggle.

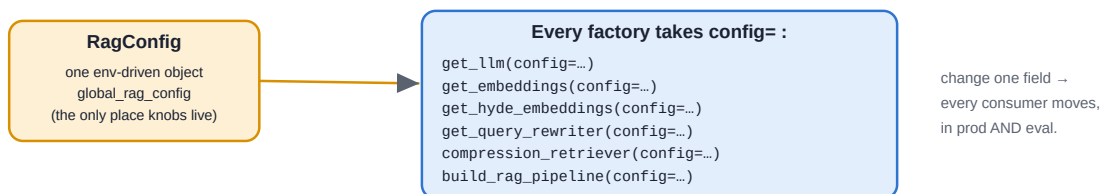
Key idea. Reranking is only useful because the dense stage now fetches rerank_fetch_k (wide) and the cross-encoder narrows to retrieval_top_k. Hybrid needs a corpus (BM25 is lexical) — production has none, so it warns and falls back to dense; evaluation passes the corpus, so hybrid genuinely runs there.

7 • The three chokepoints

The whole system is deliberately funnelled through three things you can hold in your head at once. Learn these and nothing is a mystery.

5 • The three chokepoints you must hold in your head

A • RagConfig → reaches every component through a real config= seam



B • RagTrace → filled once per run → read by every debug surface (one schema)



C • build_rag_pipeline → the only place retrieval logic lives (see diagram 4). Edit it once; production and evaluation both change.

Master these three — all knobs, all retrieval, all observability — and you can scale, debug, and fix any part of the system.

Diagram 5: A: one config reaches everything. B: one trace is read by every debug surface. C: one retrieval function.

- **A — RagConfig** is the only place knobs live, and it reaches every component through a real config= seam.

- **B — RagTrace** is filled once per run and read identically by the `/ask debug` flag, `scripts/debug_ask.py`, and the chat UI. One schema, no drift.
- **C — build_rag_pipeline** is the only place retrieval logic lives. Edit it once; production and evaluation both change.

8 · Evaluation — “what we evaluate is what we ship”

The eval harness (`tests/rag_evaluation/`) builds an in-memory index over a synthetic Q&A set and runs an ablation grid of 9 variants. Crucially, each variant now retrieves via the **production** `build_rag_pipeline` and generates with the **production** `RAG_PROMPT`; its flags are translated into a real `RagConfig`. The `production_default` row is **derived from** `global_rag_config`, so the headline number always reflects the deployed configuration.

6 · Eval == ship — evaluation drives the production code

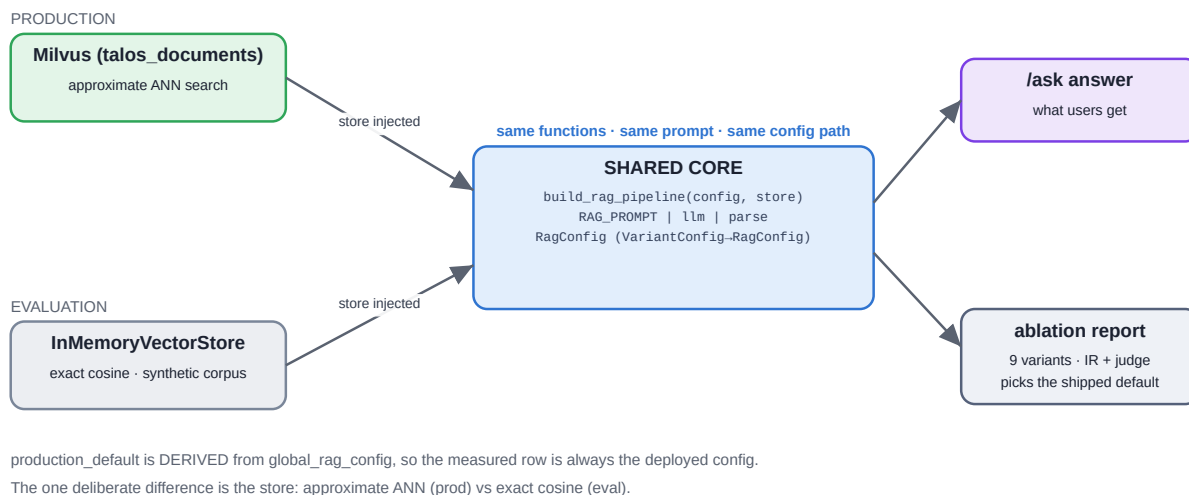


Diagram 6: Production and evaluation meet at the shared core. The only difference is the vector store.

The workflow to pick defaults is data-driven: run the grid → the winning variant's flags **become** the shipped `config.py` defaults. Metrics include IR (Hit/Recall/Precision/MRR/nDCG@k), LLM-judge (faithfulness, relevancy, correctness), bootstrap CIs, and paired significance tests.

9 · Debugging playbook

Every query fills `chain.trace` (a `RagTrace`): model, effective config, rewritten query, file/chat candidates, injected-tail size, final context, and the **exact prompt**. Read it three ways: send `{"debug": true}` to `/ask` (it appends `__ASK_DEBUG__` + JSON to the stream), run `scripts/debug_ask.py <channel> "<question>"`, or use the chat UI's 🔍 toggle.

Symptom	Check the trace / logs → likely cause
Empty / “can’t answer”	<code>file_candidates</code> empty → nothing ingested, wrong <code>workspace_id</code> , or legacy rows missing the source key (re-ingest).
Chat not remembered	<code>chat_candidates</code> empty → messages still in tier 1 (not indexed yet), indexer lagging (warn log), or recall failed (warn log with <code>chatroom_id</code>).
Milvus dimension error	Embedding provider/model changed against a populated collection → drop + re-ingest.
Hybrid “does nothing” in prod	Expected — no BM25 corpus in prod; use evaluation to measure hybrid.
Slow answers	HyDE + rewrite = two extra LLM calls per query → turn off via <code>USE_HYDE</code> / <code>USE_QUERY_REWRITE</code> .

10 · Recipes — how to change it

- **Flip a feature:** set the env var (USE_HYDE=false, USE_RERANKING=false, USE_HYBRID_RETRIEVAL=true) or change the default in `config.py`.
- **Swap model / embeddings:** OPENAI_MODEL, EMBEDDING_PROVIDER / EMBEDDING_MODEL. New provider → `extend_build_embeddings` in `vector_store.py`.
- **Change the collection:** MILVUS_COLLECTION_NAME — app and CLI both follow it.
- **Add a retrieval stage** (MMR, a second filter, ...): edit `build_rag_pipeline` once; production and evaluation both get it, and eval will measure it.
- **Tune rerank recall:** raise RERANK_FETCH_K (wider pool) vs RETRIEVAL_TOP_K (final count).
- **Change the prompt:** `src/config/prompts.py` (RAG_PROMPT) — eval uses the same object.
- **Re-pick defaults with data:** run the eval grid, set `config.py` to the winner.

Three chokepoints: **RagConfig** · **build_rag_pipeline** · **RagTrace**.